

Considered there are N philosophers seated around a circular table with one chopstick between each pair of philosophers. There is one chopstick between each philosopher. A philosopher may eat if he can pick up the two chopsticks adjacent to him. one chopstick may be picked up by any one of its adjacent followers but not both. Write a program to solve the problem using process synchronization

Code:

```
GNU nano 7.2 dinning.c *
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5 // Number of philosophers

sem_t chopstick[N];
pthread_mutex_t mutex;

void *philosopher(void *arg)
{
    int id = *(int *)arg;

    while (1)
    {
        printf("Philosopher %d is thinking\n", id);
        sleep(1);

        pthread_mutex_lock(&mutex); // enter critical section

        sem_wait(&chopstick[id]); // pick left chopstick
        sem_wait(&chopstick[(id + 1) % N]); // pick right chopstick

        printf("Philosopher %d is eating\n", id);
        sleep(2);

        sem_post(&chopstick[id]); // put left chopstick
        sem_post(&chopstick[(id + 1) % N]); // put right chopstick

        pthread_mutex_unlock(&mutex); // leave critical section
    }
}

int main()
{
    pthread_t philosophers[N];
    int id[N];

    pthread_mutex_init(&mutex, NULL);

    for (int i = 0; i < N; i++)
        sem_init(&chopstick[i], 0, 1);

    for (int i = 0; i < N; i++)
    {
        id[i] = i;
        pthread_create(&philosophers[i], NULL, philosopher, &id[i]);
    }

    for (int i = 0; i < N; i++)
        pthread_join(philosophers[i], NULL);

    return 0;
}
```

Output :

```
laxmi@laxmi: ~  
laxmi@laxmi:~$ nano dinning.c  
laxmi@laxmi:~$ gcc dinning.c -o dinning -pthread  
laxmi@laxmi:~$ ./dinning  
Philosopher 0 is thinking  
Philosopher 4 is thinking  
Philosopher 3 is thinking  
Philosopher 1 is thinking  
Philosopher 2 is thinking  
Philosopher 0 is eating  
Philosopher 0 is thinking  
Philosopher 3 is eating  
Philosopher 3 is thinking  
Philosopher 4 is eating  
Philosopher 4 is thinking  
Philosopher 1 is eating  
Philosopher 1 is thinking  
Philosopher 2 is eating  
Philosopher 2 is thinking  
Philosopher 0 is eating  
Philosopher 0 is thinking  
Philosopher 3 is eating  
Philosopher 3 is thinking  
Philosopher 4 is eating  
Philosopher 4 is thinking  
Philosopher 1 is eating  
Philosopher 1 is thinking  
Philosopher 2 is eating  
Philosopher 2 is thinking  
Philosopher 0 is eating  
Philosopher 0 is thinking  
Philosopher 3 is eating  
Philosopher 3 is thinking  
Philosopher 4 is eating  
Philosopher 4 is thinking  
Philosopher 1 is eating  
Philosopher 1 is thinking  
Philosopher 2 is eating  
Philosopher 2 is thinking  
Philosopher 0 is eating  
Philosopher 0 is thinking
```